

Reliability Testing Documentation

Dokumentasi pengujian reliability pada arsitektur microservices menjelaskan proses, skenario, metode, hasil, dan evaluasi pengujian untuk memastikan setiap layanan tetap stabil, konsisten, dan tersedia meskipun terjadi gangguan pada sebagian komponen. Dokumentasi ini berfungsi sebagai bukti bahwa mekanisme keandalan seperti retry logic, circuit breaker, health check, failover, dan monitoring telah diuji serta berjalan sesuai kebutuhan, sekaligus menjadi referensi bagi tim dalam evaluasi, pemeliharaan, audit, dan pengembangan sistem di masa mendatang.

Test Environment

Component	Environment
Frontend	Docker Compose
Gateway	Nginx
Auth Service	FastAPI
Item Service	FastAPI
Database	PostgreSQL
Container Runtime	Docker Desktop
Orchestration	Docker Compose

1. Objective

Dokumentasi ini bertujuan untuk memverifikasi kemampuan sistem microservices dalam menangani kegagalan layanan (service failure), melakukan pemulihan layanan (service recovery), serta memastikan mekanisme reliability seperti Retry Logic, Circuit Breaker, dan Aggregated Health Check berjalan sesuai harapan.

2. Retry Logic Testing

Test Scenario

Mensimulasikan kondisi ketika Auth Service tidak tersedia dan mengamati apakah Item Service melakukan retry request sebelum mengembalikan error.

Test Steps

```
docker compose stop auth-service
```

Kirim request yang membutuhkan verifikasi ke Auth Service.

Expected Result

- Item Service melakukan retry otomatis.
- Retry dilakukan sebanyak 3 kali.
- Menggunakan exponential backoff.
- Setelah retry gagal, service mengembalikan response error.

Actual Result

Belum diuji

Status

- ☐ PASS
- ☐ FAIL

Evidence

Tambahkan screenshot hasil log retry pada bagian ini.

3. Circuit Breaker Testing

Test Scenario

Memastikan Circuit Breaker dapat mendeteksi kegagalan berulang dan menghentikan request ke service yang bermasalah.

Test Steps

```
docker compose stop auth-service
```

Kirim request berulang kali ke endpoint yang membutuhkan Auth Service.

Expected Result

- Circuit Breaker berpindah ke state OPEN.
- Request berikutnya langsung ditolak.
- Tidak terjadi timeout berulang.

Actual Result

Belum diuji

Status

- ☐ PASS
- ☐ FAIL

Evidence

Tambahkan screenshot log Circuit Breaker atau response error.

4. Service Recovery Testing

Test Scenario

Memastikan sistem dapat kembali beroperasi normal setelah service yang gagal dinyalakan kembali.

Test Steps

```
docker compose start auth-service
```

Tunggu hingga service kembali sehat.

Expected Result

- Circuit Breaker berpindah ke HALF_OPEN.
- Request percobaan berhasil.
- Circuit Breaker kembali ke CLOSED.
- Sistem kembali beroperasi normal.

Actual Result

Belum diuji

Status

- ☐ PASS
- ☐ FAIL

Evidence

Tambahkan screenshot log recovery service.

5. Integration Testing

Test Scenario

Menjalankan seluruh integration test untuk memverifikasi komunikasi antar service.

Test Command

```
pytest tests/integration/ -v
```

Expected Result

Minimal 6 test berhasil.

Target ideal:

```
8 PASSED
```

Actual Result

Belum diuji

Status

- ☐ PASS
- ☐ FAIL

Evidence

Tambahkan screenshot hasil execution pytest.

6. Aggregated Health Check Testing

Test Scenario

Memastikan endpoint health menampilkan status service dan dependency yang digunakan.

Endpoint

```
GET /health
```

Expected Result

Response health menampilkan:

```
{  
  "status": "healthy",  
  "dependencies": {
```

```
"auth-service": {  
  "status": "available"  
}  
}
```

Actual Result

Belum diuji

Status

- ☐ PASS
- ☐ FAIL

Evidence

Tambahkan screenshot response endpoint health.

7. Reliability Test Summary

Scenario	Expected Result	Status
Retry Logic	Retry 3 kali sebelum gagal	⌚ Pending
Circuit Breaker	OPEN ketika service gagal berulang	⌚ Pending
Service Recovery	HALF_OPEN → CLOSED	⌚ Pending
Integration Testing	Minimal 6 test pass	⌚ Pending
Aggregated Health Check	Dependency status muncul	⌚ Pending

8. Testing Evidence

| Testing Activity | Evidence | Description |

| ----- | ----- | -----
- |

| Retry Logic Testing | (*Belum diuji*) | Tabel disiapkan untuk dokumentasi hasil pengujian Retry Logic. |

| Circuit Breaker Testing | (*Belum diuji*) | Tabel disiapkan untuk dokumentasi hasil pengujian Circuit Breaker.
|

| Service Recovery Testing | (*Belum diuji*) | Tabel disiapkan untuk dokumentasi hasil pengujian pemulihan layanan. |

| Integration Testing | (*Belum diuji*) | Tabel disiapkan untuk dokumentasi hasil pengujian integrasi antar service. |

| Aggregated Health Check | (*Belum diuji*) | Tabel disiapkan untuk dokumentasi hasil pengujian endpoint health check. |

9. Conclusion

Pengujian reliability dilakukan untuk memastikan arsitektur microservices mampu menangani kondisi kegagalan layanan tanpa menyebabkan seluruh sistem berhenti beroperasi.

Final Status

 Reliability Testing In Progress